What Does a Fixed Model Gain from the System Around It?
In-task checks and organisational changes for Gemma 4 coding agents

Ghaisan Khoirul Badruzaman
Independent Researcher

November 10, 2026

Abstract

We study whether changing how a fixed small model is run makes it more capable. The Gemma 4 Developer Agent Competition fixes the harness and the time budget; we also fix the model (Gemma 4 31B, no adapter), so only the agent's organisation, the system around the model, changes. First, we evaluate the agent's claim of completion, its own reproduction script, a fresh-context judge and nearby existing tests as four in-task checks against the hidden grading tests. The bare claim has the highest pooled likelihood ratio (LR 1.74 against 1.13 and 1.08 for the executed checks), but every timed-out patch is wrong and all but one are unclaimed. Among finished sessions the claim accepts every wrong patch (LR 0.73), and executed checks are uninformative (LR 1.0). Only a fresh-context judge remains informative (LR 1.30). We prove that selecting among n candidates with a check of ratio LR adds at most $p(\mathrm{LR}-1)$ to a model with success p. Second, we test five organisational changes, four at equal budget, in task-paired comparisons on 71 development tasks and the 62 post-cutoff tasks of AXI-Fresh. A two-stage role split lost 19 tasks and won none, and a five-role pipeline scored one task below the single loop on the leaderboard. A shared default stack and a parser-safe edit form had no detectable effect, and session-state notes added to both resolved 24 of 71 against 31. Only time-budget governance is favoured by every comparison (+1 of 58 on the leaderboard, +3 of 62 on AXI-Fresh and +12 of 71 on the development tasks), but its one registered test (AXI-Fresh, across hardware) failed ($p = 0.32$).

1 Introduction

Research questions. Compound AI systems change how a fixed model is run [1]. Whether this helps at an *equal* budget is open for software tasks; on reasoning tasks a single agent performs comparably to multi-agent systems at the same thinking-token budget [2]. (1) How informative are an agent's own checks (Sections 3 and 4)? (2) Which organisational change resolves more unseen tasks than a single loop at equal time and tool budget: roles, defaults, the edit form, session-state notes or time-budget governance (Section 4)?

Setting. The competition fixes the harness and twelve hours for about 120 private tasks (about six minutes each) and accepts four Gemma 4 sizes and LoRA adapters. We use Gemma 4 31B with a 32k-token context and no adapter (supplement S2[1]). The *grading tests* are a task's hidden tests, which the agent never sees. The *public leaderboard* scores 58 of the private tasks (inferred from its score steps) as a share truncated to two decimals. The *single loop* is the host's baseline agent with our prompt fixes. The *development tasks* are 71 of the 105 public tasks whose reference

[1]The supplement (S1 to S7) is a separate PDF released with the dataset. Identifiers: `r` rows are runs and `e` rows are check analyses in the released ledger (Section 6); `P` names a pre-registration and `H`, `E` and `A` its hypotheses and amendments, each hashed before its outcome, except P6's A4: committed before the baseline rerun's first result (both 2026-11-04), its hash published after it (2026-11-09).

fix passes the competition's sandbox. A seeded split drew them and held out one third per repository (34 tasks, read by no agent). *AXI-Fresh* is a set of 62 tasks built from fixes merged in 2026 and checked in the competition's grader (Section 5).

Contributions. (1) Pre-registered predictions and measured likelihood ratios of four in-task checks against hidden grading tests (supplement S3). (2) A closed form for selecting among n candidates with a check of any sensitivity, and a proof that the gain is at most $p(\mathrm{LR}-1)$ (Propositions 1 and 2). (3) Task-paired tests of five organisational changes, four of them at equal budget (Section 4). (4) Three rules for builders of small-model agents.

Three rules.

1. Do not use an in-task check as a verifier: among finished sessions the reproduction script and nearby tests are uninformative (LR 1.0), the claim accepts every wrong patch, and a fresh-context judge lifts Proposition 1's plateau only from the finished sessions' 0.70 to 0.75 (Table 1).
2. Force a first edit early: an executed check at LR 1 costs minutes, and on tasks some configuration resolved, 46 of 61 failing sessions had seen a file that a resolving patch changes and never edited it (Section 4).
3. Plan to finish inside the cap: the harness grades the tree as it stands when time runs out, yet no session cut off by the cap resolved its task (0 of 205) against 66 of the 124 that submitted in time (descriptive).

2 Study design

In-task checks. Rules were frozen before any verdict was scored (supplement S3). *Self-claim*: the trace holds a `submit_patch` call whose observation is ok (the frozen rule A1); a session cut off by the cap without one makes no claim. *Own reproduction*: a test file the agent wrote exits 0 on its last run after the last source edit (undefined when it wrote none, 75 of 182 patches). *Fresh judge*: the same model answers accept or reject in a new context given only the issue and the diff. *Nearby tests*: up to five existing test files naming or importing a changed module are run before and after the patch; accept when no passing test breaks (undefined for 50 of 182). Sensitivity α and false acceptance β are the shares of correct and of wrong patches accepted; $\mathrm{LR} = \alpha/\beta$, and at $\mathrm{LR} = 1$ a check is uninformative.

Organisational changes. Each change is evaluated separately against its control on the same tasks. *Roles* covers a five-role pipeline we entered on the leaderboard (supplement S7) and a two-stage split tested on the development tasks, in which a read-only scout hands over to a fixer in a fresh context. The *shared default stack* holds four defaults the leading forum packages share: the statement kept in the instruction, thinking capped at 2,048 against 4,096, the host's sampling settings and no repeat of a rejected call. The *parser-safe edit form* sends the edit as one string through `run_command` instead of the structured `edit_file` call, which the parser rejected 601 of 2,057 times (supplement S2). *Time-budget governance* is the public Budget-Fit configuration [3], whose cap was swept [4] and whose edit deadline was measured [5]. It allows 8 minutes and 28 tool calls per task against 3 to 4 minutes and 50 calls before, and its prompt requires a first source edit by the 12th call. *Session-state notes* are a task-state file the agent re-reads after context compaction, tested on top of the default stack and the edit form, so its row measures all three.

3 Selection bounds and predictions

Propositions. Resampling against an imperfect verifier is capped by its false positives [6], whose worked example sets the verifier's sensitivity to one. For a check with any sensitivity α and false acceptance β, Propositions 1 and 2 give the plateau in closed form and bound the gain by the likelihood ratio (proofs in supplement S1). *Proposition 1.* Let n candidates be independently correct with probability p, let the check's verdicts be conditionally independent given correctness ($\alpha + \beta > 0$), and let the agent pick uniformly among accepted candidates (among all if none is accepted). Then its success is

$$P_n = q - (q-p)\,a^{n-1}, \qquad a = 1 - p\alpha - (1-p)\beta, \qquad q = \frac{p\alpha}{p\alpha + (1-p)\beta}.$$

Selection plateaus at q, whose odds are LR times p's. *Proposition 2.* If $\alpha \geq \beta > 0$, then for every n

$$P_n - p \leq p(\mathrm{LR} - 1),$$

so a check with LR near one adds almost nothing for any number of candidates. If some of an author's errors are blind spots shared by any judge with the same information [7, 8], a judge with different information should catch some (Proposition 3, supplement S1). H6 tests the weakest version by giving the same model a fresh context with less information.

Table 1: In-task checks against the hidden grading tests on pool v1 (182 patches from 60 development tasks, 89 of them correct; e0002), with 90% bootstrap intervals over tasks. Judged: patches on which the check's verdict is defined. Row 5 is exploratory and recomputed by the public notebook. Last block: the 127 patches from finished sessions (e3-terminal-01, post hoc; intervals over patches).

check	judged	sensitivity α	false acceptance β	LR [90%]
self-claim	182	0.73	0.42	1.74 [1.37, 2.30]
own reproduction	107	0.95	0.84	1.13 [1.01, 1.27]
fresh judge	182	0.96	0.62	1.53 [1.28, 1.94]
nearby tests	132	1.00	0.93	1.08 [1.02, 1.15]
self-claim and fresh judge, both accept	182	0.71	0.31	2.27 [1.63, 3.39]
finished sessions only: self-claim	127	0.73	1.00	0.73 [0.65, 0.81]
fresh judge	127	0.96	0.74	1.30 [1.12, 1.56]
both accept	127	0.71	0.74	0.96 [0.80, 1.18]

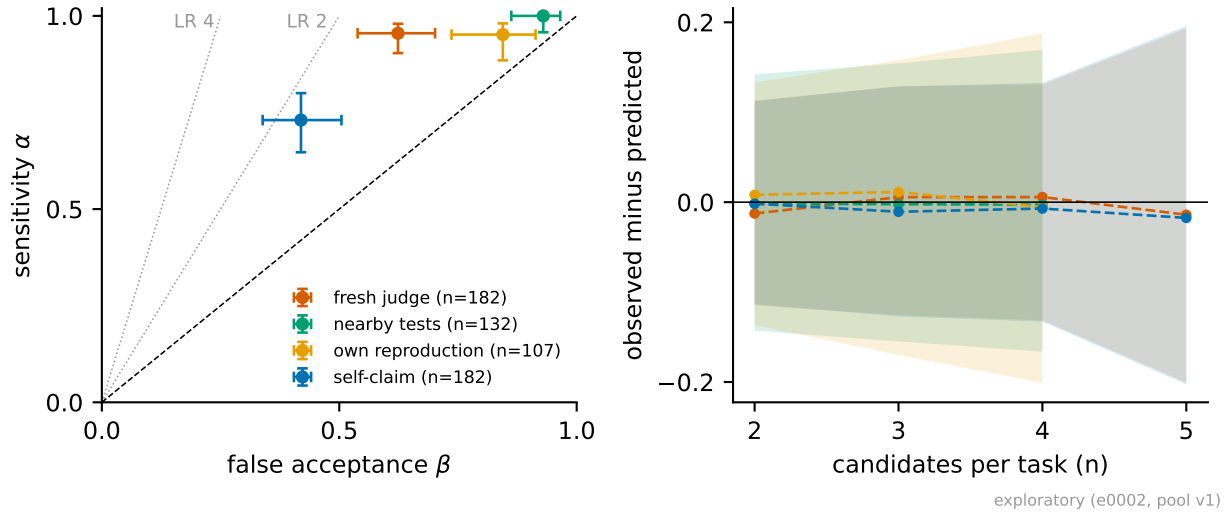


Figure 1: Left: false acceptance and sensitivity per check (90% intervals); the diagonal marks LR 1. Right: observed best-of-n success minus Proposition 1's prediction on the same tasks, with the prediction's 90% band; exploratory, pool v1 (e0002).

Patch pool and predictions. The checks are measured on pool v1: 182 non-empty patches that five single-loop configurations produced on 60 development tasks (49% correct; r0005 to r0007, r0009, r0010; e0002). The configurations differ in prompt, thinking and time cap (3 to 4 minutes, 50 calls). P5 registered every development patch up to the freeze, but the fresh judge ran once, so the analysis stops at pool v1 (a deviation; supplement S3). Its verdicts came after an interim look at the other three checks, so H6 is confirmatory and the other predictions exploratory (amendment A9). Registered before any outcome: the executed checks exceed the self-claim's LR (*E3-a*); the fresh judge lets fewer wrong patches through than the self-claim (*H6*); best-of-n success follows Proposition 1 (*H3*); the pipeline outperforms the single loop on AXI-Fresh at equal budget (*H-Sys*, P6).

4 Results

Likelihood ratios. Table 1 and Figure 1 give each check's rates. Executed checks have lower pooled ratios than the agent's own claim (the self-claim recomputed on the same patches remains higher, 1.34 against 1.13 and 1.67 against 1.08), so E3-a is refuted (exploratory, pool v1). The pooled ratios reflect timeouts because the 55 patches from timed-out sessions are all wrong and 54 of them carry no claim. Among the 127 patches from sessions that finished, the self-claim accepts every wrong patch (LR 0.73). Own reproduction and nearby tests are uninformative there (0.99, 1.03). The fresh judge remains weakly informative (1.30) and the conjunction falls to 0.96 (analysis e3-terminal-01, exploratory, post hoc).

Table 2: Task-paired tests (development: 71 tasks; AXI-Fresh: 62). Every pair shares hardware and rollout version except the time-budget pair. b, c: tasks resolved by the change only, by the control only; p: one-sided sign-flip test that the change resolves more (p near 1: the discordant tasks favoured the control). Detectable lift at 80% power 0.18 and 0.26 (supplement S4).

change	set, arm/control	resolved	b	c	p
roles: two-stage split	development, r0052/r0049	12 against 31	0	19	1.00
session-state notes, with defaults and edit form	development, r0053/r0049	24 against 31	3	10	0.99
shared default stack	development, r0050/r0049	31 against 31	4	4	0.64
parser-safe edit form	AXI-Fresh, r0044/r0045	44 against 46	9	11	0.75
time-budget governance	AXI-Fresh, r0045/r0054 (P6 after A4, across hardware)	46 against 43	11	8	0.32

Fresh judge against self-claim (H6). H6 predicted a fresh-context judge would let fewer wrong patches through than the author's claim; pooled data contradict it. Of the 93 wrong patches both verdicts covered (confirmatory, A9), the fresh judge accepted 29 that the self-claim had rejected and rejected 10 that it had accepted (one-sided McNemar test in the predicted direction, $p = 0.9995$). Requiring both to accept reaches LR 2.27 pooled and 0.96 among finished sessions, where the registered direction holds (judge false acceptance 0.74 against the claim's 1.00). A fresh context alone therefore did not make the same model a stricter judge. Best-of-n success falls within Proposition 1's in-sample prediction for $n = 2$ to 5 (H3; Figure 1, right), and no observed lift exceeds Proposition 2's bound (E3-b).

Single-loop failure modes. Across the five single-loop configurations of the pool, 205 of 355 sessions hit the cap and none of the 90 resolutions came from one of them (66 from the 124 finished sessions that submitted, 24 from the 26 that stopped without submitting). Most failing sessions never edited a source file (179 of 265; analysis trace-fail-01). On the 30 tasks some configuration resolved with a non-empty patch, 46 of the 61 failing sessions had seen a file that a resolving patch changes and never edited it; 40 of those timed out.

Organisational changes. Table 2 applies the exact one-sided paired sign-flip test at the 0.10 level fixed at pre-registration.

Roles. The five-role pipeline scored 7 of 58 on the leaderboard (2026-10-07) against 8 for that week's single loop; the pipeline was never re-run against the fitted budget. The two-stage variant resolved 12 of 71 against 31, and all 19 discordant tasks favoured the single loop. The pre-registered H-Sys pipeline arm was never obtained (its first run lost its model server at the tenth task, r0015, r0016).

Defaults and edit form. The shared default stack tied its control (31 against 31) and the parser-safe edit form lost two tasks on AXI-Fresh (44 against 46). Session-state notes that survive context compaction, added to both, resolved 24 against 31 (3 gained, 10 lost; r0053).

Time-budget governance. With Budget-Fit the single loop resolved 31 of 71 development tasks (r0049) against 19 for the baseline with prompt fixes (r0010; 13 gained, 1 lost). Because the harness versions differ, this comparison is descriptive; the leaderboard agrees (9 to 11 against 8 of 58). On AXI-Fresh the registered test (P6 after amendment A4) pairs Budget-Fit (r0045) with the prompt-fix baseline (r0054): 46 against 43 of 62 ($b = 11$, $c = 8$, one-sided $p = 0.32$). A4 named r0045 when its result was known and the baseline rerun's (r0054) was not, so this tests time-budget governance and leaves H-Sys untested. P6 predicted 16 to 26 for the baseline and called the claim dead at 40 or more; the baseline resolved 43, so the registered claim fails. The arms also cross hardware and rollout versions (supplement S2).

5 Related work

Verifiers. Benchmarks grade a patch with hidden tests [9], and a passing patch can still be wrong [10]. Choosing among samples needs a verifier trained [11] or assembled from weak signals [12]. Stroebl et al. [6] show that false positives cap the gain from resampling, and Proposition 2 bounds that gain for a verifier of any sensitivity. Reproduction tests select patches well for large models [13]; here they are uninformative. Competitors published grader soundness controls [14] and an unquantified remark that a passing reproduction probe does not predict a passing hidden test [15]. Table 1 quantifies that remark as a likelihood ratio. Models favour their own generations [8] and share blind spots with judges holding the same information [7]; H6 tests that.

Organisation and fresh task sets. Role-split agents [16, 17] report gains with large models. At small scale a competitor found no lift from two agents [14], and Liu [18] found that weak managers directing weak workers scored below a single agent. Our two-stage split adds a third negative result. Post-cutoff task sets exist [19–21], and our companion resource AXI-Fresh is one built and checked in this competition's grader (https://kaggle.com/competitions/gemma-4-developer-agent-paper/writeups/axi-fresh-post-cutoff-tasks).

6 Limits and implications

Threats to validity. (1) The five-role result rests on one leaderboard score. The decomposition tests each change alone, so interactions are untested, and Budget-Fit changes its prompt together with its limits. Each arm is a single run; two runs of one Budget-Fit bundle on 29 shared AXI-Fresh tasks resolved 25 and 22 (r0047, r0045; rollout versions differ). (2) In both arms AXI-Fresh runs without the harness's code-graph tools, which the private leaderboard provides. (3) The development tasks come from FastAPI (42), Rich (28) and Requests (1), and AXI-Fresh from 22 small repositories. All check ratios are from Gemma 4, and the public-trajectory results are descriptive.

Generalisability. The measurement (any agent with logs and grading tests), Propositions 1 and 2 (any check) and the paired equal-budget design do not depend on Gemma 4. For any agent under a cap, pooled ratios mix judgement with finishing in time, so a check should be read among finished sessions. The rules are measured on this model only, and which organisational change helps depends on its speed and on this harness's parser. The pooled ordering recurs on public trajectories of OpenHands with Qwen3-Coder-480B [22] and SWE-agent [23] with Llama 3.1 [24]: self-claim LR 1.20 and 1.62, own reproduction 1.08 and 1.56 (supplement S5; exploratory, pooled over exits).

Data and code. The Kaggle dataset `ghaisank/axi-paper-release` (mirrored at `github.com/Finerium/axi-paper-release`) holds `ledger-public.jsonl` (holdout task ids omitted), `e3-verdicts.csv`, `e3-session-exit.csv`, `p6-tasks.csv`, `public-trajectories.csv` and the supplement, and a public notebook recomputes Tables 1 and 2 from them (https://www.kaggle.com/code/ghaisank/axi-paper-recompute). The pre-registration texts are in the release dataset and their hashes in `ghaisank/axi-preregistration-hashes`. Agent code is withheld until the main track closes (2026-12-02); both tables can be recomputed without it.

Use of AI. The author set the questions, the pre-registered predictions and the rules of the study (supplement S6), read every draft and all 62 released task statements and is responsible for every claim. Under those rules a Claude Code agent ran the experiments and drafted this text.

References

[1] Matei Zaharia et al. The Shift from Models to Compound AI Systems. Berkeley Artificial Intelligence Research (BAIR) Blog, 18 February 2024, 2024. URL https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/. Accessed 2026-10-06.

[2] Dat Tran and Douwe Kiela. Single-agent LLMs outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets. *arXiv:2604.02460*, 2026.

[3] Wilmer E. Henao. Gemma 4 agent: Budget-fit single agent. Kaggle notebook verracodeguacas/gemma-4-agent-budget-fit-single-agent, Gemma 4 Developer Agent Competition, 2026-10-05, 2026. URL https://www.kaggle.com/code/verracodeguacas/gemma-4-agent-budget-fit-single-agent.

[4] Ayush Thakur. Gemma 4 agent experiments: sampling, thinking, time limits, and tool ablations. Kaggle discussion 746250, Gemma 4 Developer Agent Competition, 2026-10-06, 2026. URL https://www.kaggle.com/competitions/gemma-4-developer-agent/discussion/746250.

[5] kimdonggyu1255. Found, not fixed: 38% of default-prompt tasks never edit a source file; an edit deadline turns most into wrong edits. Kaggle discussion 749198, Gemma 4 Developer Agent Competition, 2026-10-14, 2026. URL https://www.kaggle.com/competitions/gemma-4-developer-agent/discussion/749198.

[6] Benedikt Stroebl et al. The Limits of Inference Scaling Through Resampling. *arXiv:2411.17501*, 2024.

[7] Ken Tsui. Self-Correction Bench: Uncovering and Addressing the Self-Correction Blind Spot in Large Language Models. *arXiv:2507.02778*, 2025.

[8] Arjun Panickssery et al. LLM Evaluators Recognize and Favor Their Own Generations. *arXiv:2404.13076*, 2024.

[9] Carlos E. Jimenez et al. Swe-bench: Can language models resolve real-world github issues? *arXiv:2310.06770*, 2023.

[10] You Wang et al. Are "Solved Issues" in SWE-bench Really Solved Correctly? An Empirical Study. *arXiv:2503.15223*, 2025.

[11] Naman Jain et al. R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents. *arXiv:2504.07164*, 2025.

[12] Jon Saad-Falcon et al. Shrinking the Generation-Verification Gap with Weak Verifiers. *arXiv:2506.18203*, 2025.

[13] Chunqiu Steven Xia et al. Agentless: Demystifying llm-based software engineering agents. *arXiv:2407.01489*, 2024.

[14] Dmitrii Gluzdov. Gemma 4: Task Soundness, Serving and Evaluation. Kaggle notebook, 2026. URL https://www.kaggle.com/code/dmitriigluzdov/gemma-4-task-soundness-serving-and-evaluation. Accessed 2026-11-10.

[15] Agent Trainer. Agentic | Counterexample Lab. Kaggle notebook, 2026. URL https://www.kaggle.com/code/lucifer19/agentic-counterexample-lab. Accessed 2026-10-05.

[16] Dong Chen et al. CodeR: Issue Resolving with Multi-Agent and Task Graphs. *arXiv:2406.01304*, 2024.

[17] Daman Arora et al. MASAI: Modular Architecture for Software-engineering AI Agents. *arXiv:2406.11638*, 2024.

[18] Rui Liu. Can AI Models Direct Each Other? Organizational Structure as a Probe into Training Limitations. *arXiv:2603.26458*, 2026.

[19] Ibragim Badertdinov et al. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. *arXiv:2505.20411*, 2025.

[20] Linghao Zhang et al. SWE-bench Goes Live! *arXiv:2505.23419*, 2025.

[21] jerrydjin. unseen52 offline SWE split. Kaggle dataset, 2026. URL https://www.kaggle.com/datasets/jerrydjin/unseen52-offline-swe. Accessed 2026-10-14.

[22] Nebius. SWE-rebench-openhands-trajectories. Hugging Face dataset, CC BY 4.0, 2025. URL https://huggingface.co/datasets/nebius/SWE-rebench-openhands-trajectories. Accessed 2026-10-06.

[23] John Yang et al. Swe-agent: Agent-computer interfaces enable automated software engineering. *arXiv:2405.15793*, 2024.

[24] Nebius. SWE-agent-trajectories. Hugging Face dataset, CC BY 4.0, 2024. URL https://huggingface.co/datasets/nebius/SWE-agent-trajectories. Accessed 2026-10-06.